Lecture 2
Basic Neural Networks I

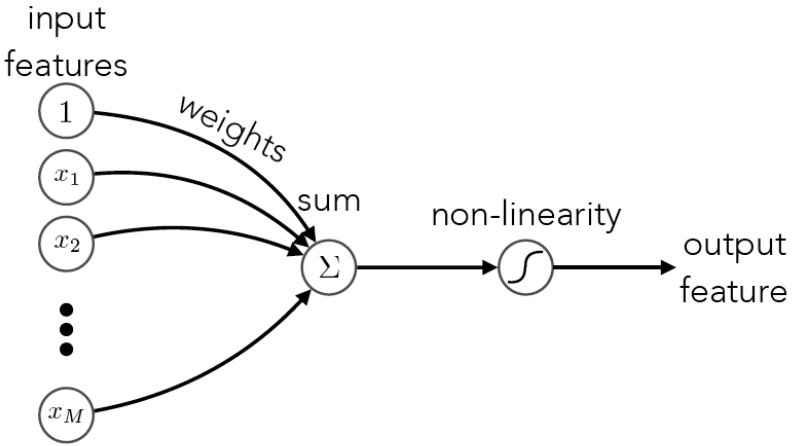
Ying CAO
SIST, ShanghaiTech
Spring, 2026

Outline

- Artificial neuron
  - Perceptron algorithm
- Single-layer neural networks
  - Network models
  - Multi-class classification

Acknowledgement: Hugo Larochelle's, Mehryar Mohri's & Yingyu Liang's course notes

Mathematical model of a neuron



artificial neuron: *weighted sum and non-linearity*

bias — input features

$$s = b + w_1 x_1 + w_2 x_2 + \cdots + w_M x_M = \mathbf{w}^\intercal \mathbf{x}$$

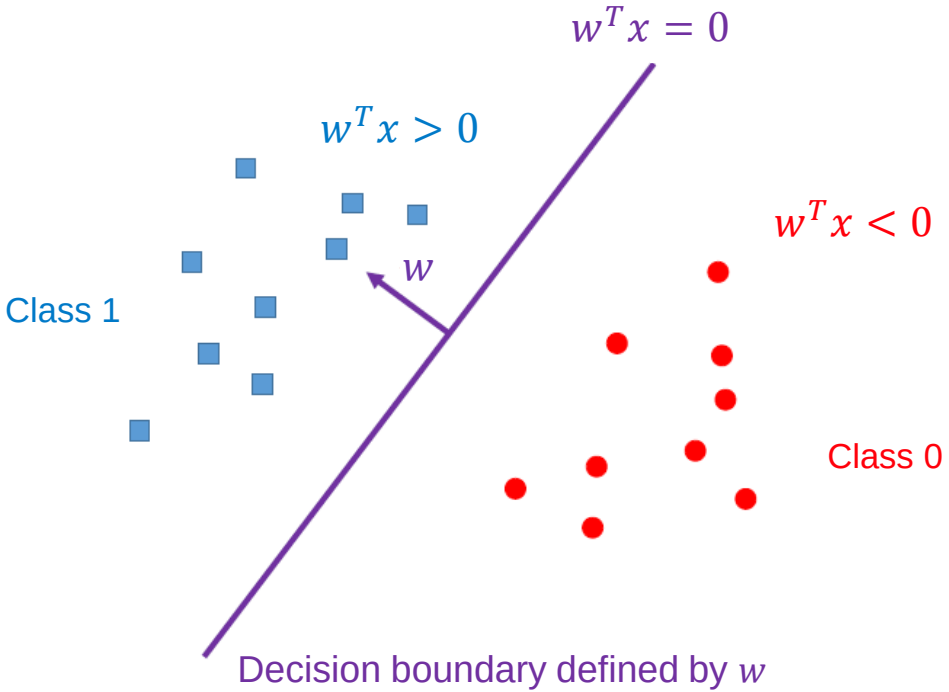
sum — weights

$$h = g(s)$$

output feature — non-linearity — sum

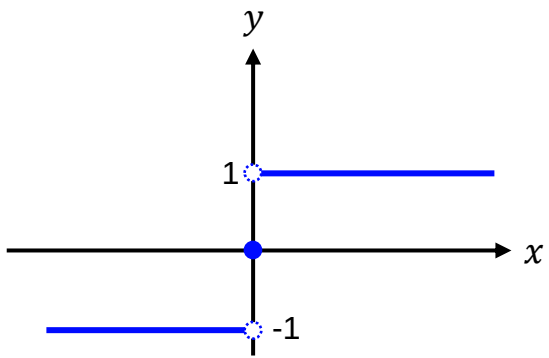
Single neuron as a linear classifier

- Perceptron model for binary classification



How to determine weights?

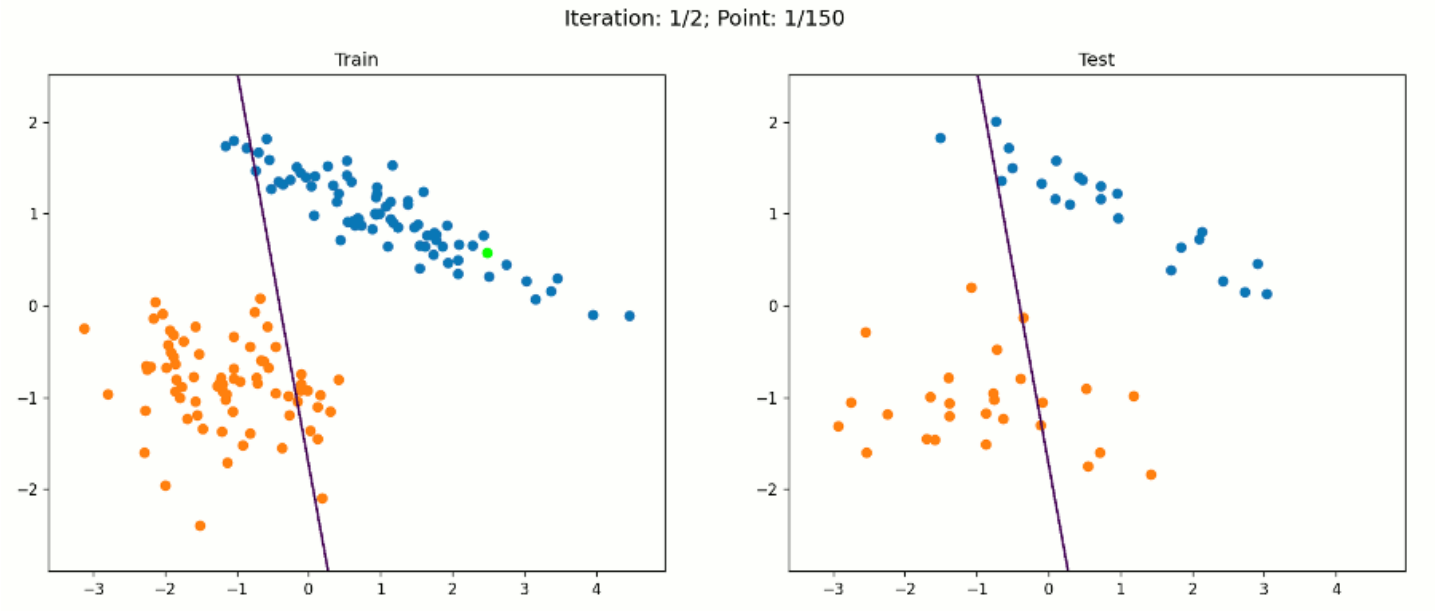
- Learning problem:
  - Given training data $\{(x_i, y_i)\}_{i=1}^{n}$ with $y_i \in \{1, -1\}$
  - Hypothesis: $f_w(x) = w^T x$
    - $y = 1$ (positive) if $w^T x > 0$
    - $y = -1$ (negative) if $w^T x < 0$
  - Prediction: $y = \text{sign}\big(f_w(x)\big) = \text{sign}(w^T x)$
  - Goal: find w that minimizes classification error



$y = \text{sign}(x)$

Perceptron algorithm

- Learn a single neuron for binary classification



Perceptron algorithm

- Outline

1. Start with the all-zero weight vector $w_1 = 0$, and set $t = 1$

2. Given example x, predict positive if $w_t^T x > 0$; predict negative if $w_t^T x < 0$

3. On a mistake, update:
   - Mistake on positive: $w_{t+1} = w_t + x$
   - Mistake on negative: $w_{t+1} = w_t - x$

4. $t \leftarrow t + 1$, and go to step 2 until all examples are classified correctly

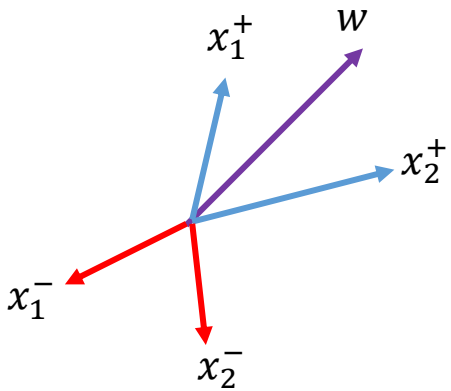
Perceptron algorithm

- Intuition behind the weight update rule

$$w^T x = \|w\|\|x\|\cos\theta$$

$y = +1$ if $w^T x > 0 \rightarrow \cos\theta > 0 \rightarrow \theta < 90$

$y = -1$ if $w^T x < 0 \rightarrow \cos\theta < 0 \rightarrow \theta > 90$



Mistake on positive: $w_{t+1} = w_t + x$

$$\cos\theta_{t+1} = \frac{w_{t+1}^T x}{\|w_{t+1}\|\|x\|} = \frac{(w_t + x)^T x}{\|w_t + x\|\|x\|} = \frac{w_t^T x + x^T x}{\|w_t + x\|\|x\|} \overset{x^T x > 0}{>} \frac{w_t^T x}{\|w_t + x\|\|x\|} = \frac{\|w_t\|\|x\|\cos\theta_t}{\|w_t + x\|\|x\|}$$

$$= \frac{\|w_t\|}{\|w_t + x\|}\cos\theta_t \geq \frac{\|w_t\|}{\sqrt{\|w_t\|^2 + \|x\|^2}}\cos\theta_t > \cos\theta_t \quad \Rightarrow \quad \boxed{\theta_{t+1} < \theta_t}$$

$\|w_t + x\|^2 = \|w_t\|^2 + \|x\|^2 + 2w_t^T x \leq \|w_t\|^2 + \|x\|^2$ $\qquad$ $\cos\theta_t \leq 0$

Perceptron algorithm

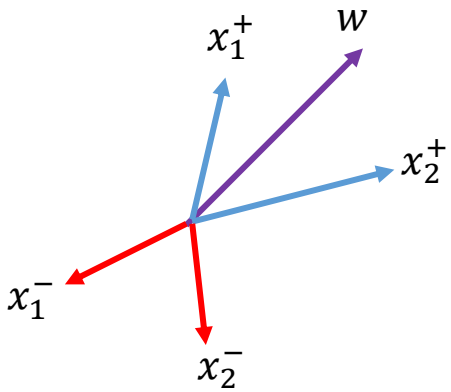
- Intuition behind the weight update rule

$$w^T x = \|w\|\|x\| \cos\theta$$

$y = +1$ if $w^T x > 0 \rightarrow \cos\theta > 0 \rightarrow \theta < 90$

$y = -1$ if $w^T x < 0 \rightarrow \cos\theta < 0 \rightarrow \theta > 90$

Mistake on negative: $w_{t+1} = w_t - x$



$$\cos\theta_{t+1} < \cos\theta_t \Rightarrow \boxed{\theta_{t+1} > \theta_t}$$

Perceptron algorithm

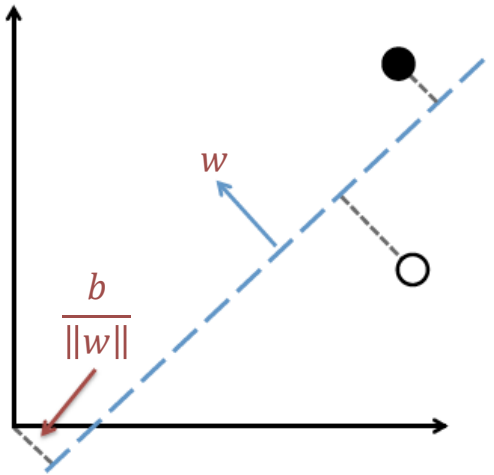
- Perceptron theorem
  - Suppose there exist w^* that correctly classifies $\{(x_i, y_i)\}$
  - W.L.O.G., all x_i and w^* have length 1, so the minimum distance of any example to the decision boundary (hyperplance $(w^*)^T x = 0$) is

$$\gamma = \min_i |(w^*)^T x_i|$$

  - Then perceptron makes at most $\left(\frac{1}{\gamma}\right)^2$ mistakes

Review: Hyperplane

- A (N-1)-D plane in N-D space
  - 1D line in 2D space, 2D plane in 3D space
  - Defined by $w^T x + b = 0$
  - Distance from x to the plane: $\frac{|w^T x+b|}{\|w\|}$
  - Signed distance: $\frac{w^T x+b}{\|w\|}$



Perceptron theorem

- Proof
  - Claim 1: $w_{t+1}^T w^* \geq w_t^T w^* + \gamma$
  - Proof:
    - If mistake on a positive example

$$w_{t+1}^T w^* = (w_t + x)^T w^* = w_t^T w^* + x^T w^* \geq w_t^T w^* + \gamma$$

Positive distance since w^* can classify x correctly

    - If mistake on a negative example

$$w_{t+1}^T w^* = (w_t - x)^T w^* = w_t^T w^* - x^T w^* \geq w_t^T w^* + \gamma$$

Perceptron theorem

- Proof
  - Claim 2: $\|w_{t+1}\|^2 \leq \|w_t\|^2 + 1$
  - Proof:
    - If mistake on a positive example

$$\|w_{t+1}\|^2 = \|w_t + x\|^2 = \|w_t\|^2 + \|x\|^2 + 2w_t^T x \leq \|w_t\|^2 + 1$$

Assume $\|x\|^2=1$

≤ 0 since we made a mistake on x

    - If mistake on a negative example

$$\|w_{t+1}\|^2 = \|w_t - x\|^2 = \|w_t\|^2 + \|x\|^2 - 2w_t^T x \leq \|w_t\|^2 + 1$$

Perceptron theorem

- Proof
  - Claim 1: $w_{t+1}^T w^* \geq w_t^T w^* + \gamma$
  - Claim 2: $\|w_{t+1}\|^2 \leq \|w_t\|^2 + 1$
  - After M mistakes:
    - $w_{M+1}^T w^* \geq \gamma M$ (Claim 1)
    - $\|w_{M+1}\| \leq \sqrt{M}$ (Claim 2)
    - $w_{M+1}^T w^* \leq \|w_{M+1}\|$ (w^* has length 1)
  - So $\gamma M \leq \sqrt{M}$ and thus $M \leq \left(\frac{1}{\gamma}\right)^2$

Perceptron algorithm

- General learning problem setup
  - Given training data $\{(x_i, y_i)\}_{i=1}^n$
  - Find $f: x \to y \in \mathcal{H}$ that minimizes $L(f) = \frac{1}{n}\sum_{i=1}^n l(f, x_i, y_i)$

- What loss function is minimized?

$$l(w, x_i, y_i) = -y_i w^T x_i \mathbb{I}[\text{mistake on } x_i]$$

1 if mistake on x_i and 0 otherwise

$$L(w) = -\frac{1}{n}\sum_{i=1}^{n} y_i w^T x_i \mathbb{I}[\text{mistake on } x_i]$$

Perceptron algorithm

- Parameter update with SGD:

$$w_{t+1} = w_t - \eta \nabla_{w_t} l(w_t, x_i, y_i) = w_t + \eta y_i x_i \mathbb{I}[\text{mistake on } x_i]$$

  - Set $\eta = 1$
  - If mistake on a positive example x_i ($y_i = 1$):

$$w_{t+1} = w_t + y_i x_i = w_t + x_i$$

  - If mistake on a negative example x_i ($y_i = -1$):

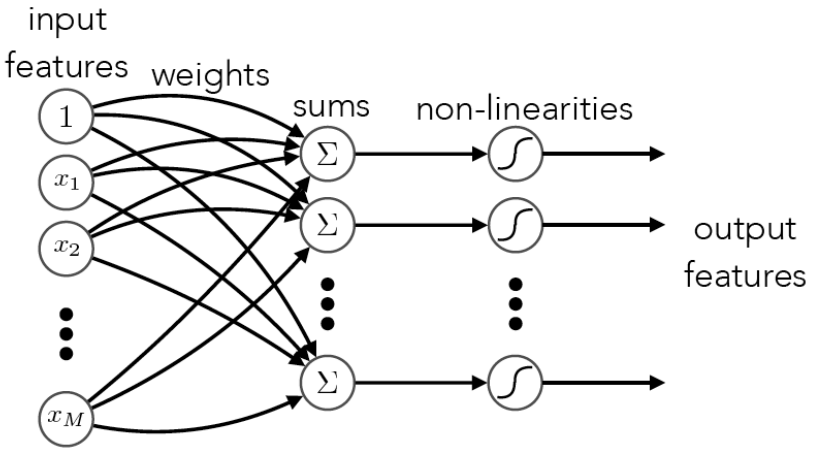
$$w_{t+1} = w_t + y_i x_i = w_t - x_i$$

Outline

- Artificial neuron
  - Perceptron algorithm
- Single-layer neural networks
  - Network models
  - Multi-class classification

Acknowledgement: Hugo Larochelle's, Mehryar Mohri's & Yingyu Liang's course notes

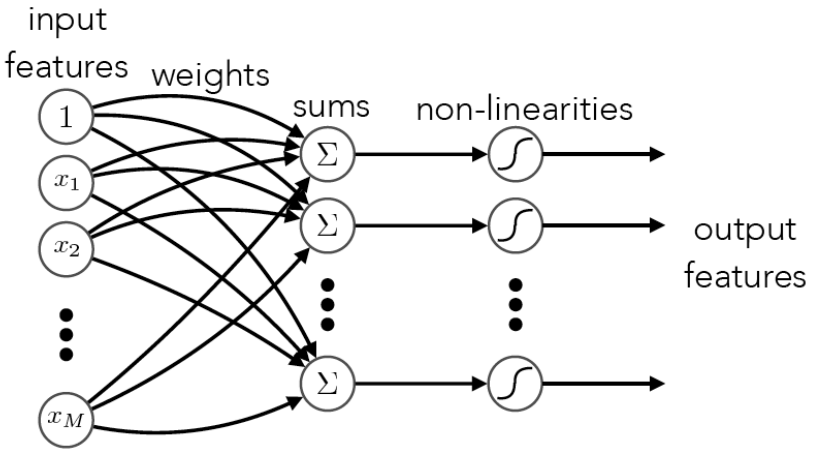
Single-layer neural network

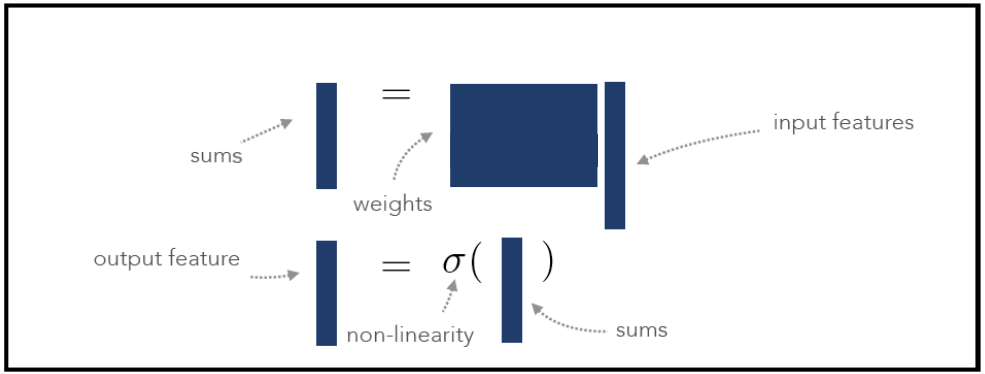


one sum per weight *vector* $s_j = \mathbf{w}_j^\intercal \mathbf{x}$ $\longrightarrow$ $\mathbf{s} = \mathbf{W}^\intercal \mathbf{x}$ vector of sums from weight *matrix*

$$\mathbf{h} = \sigma(\mathbf{s})$$

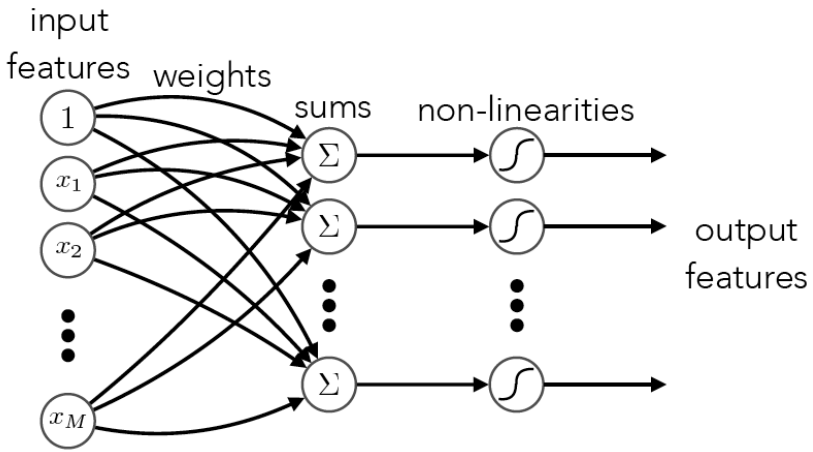
Single-layer neural network





Single-layer neural network

- Neuron-wise non-linearity
  - Independent feature detectors

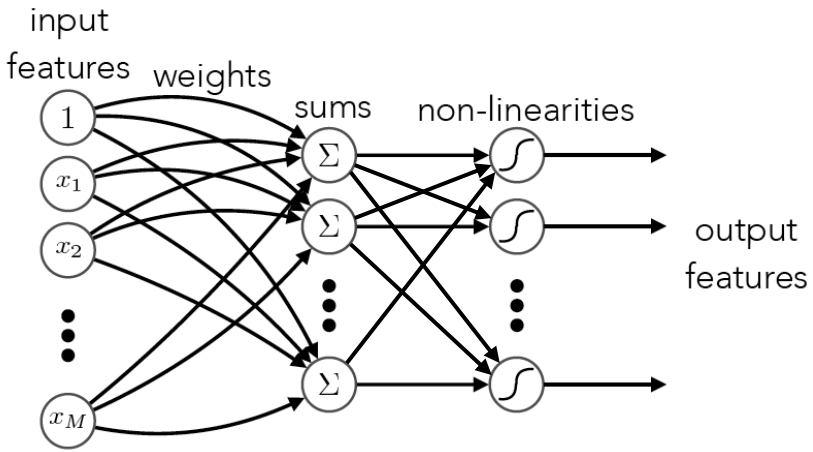


$$\mathbf{h} = [h_j]$$

$$h_j = \sigma(s_j) = \sigma(\mathbf{w}_j^\mathsf{T}\mathbf{x})$$

Single-layer neural network

- Non-linearity with all sums as input
  - Competition between neurons

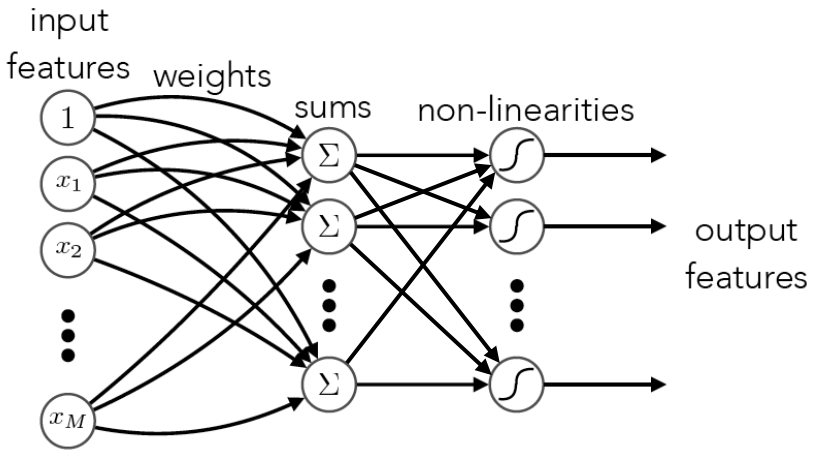


$$\mathbf{h} = [h_j]$$

$$h_j = g(\mathbf{s}) = g(\mathbf{w}_1^\intercal \mathbf{x}, \cdots, \mathbf{w}_m^\intercal \mathbf{x})$$

Single-layer neural network

- Non-linearity with all sums as input
  - Winner-Take-All (WTA) for multi-class classification



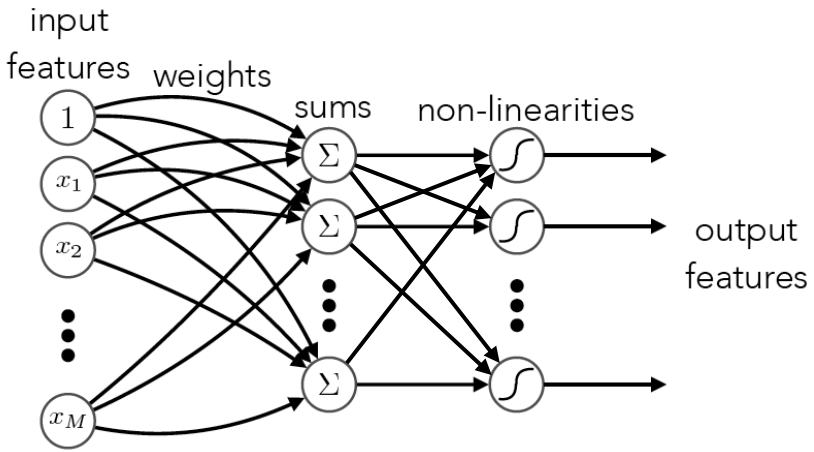
Only one neuron outputs 1; the rest output 0

$$\mathbf{h} = [h_j]$$

$$h_j = g(\mathbf{s}) = \begin{cases} 1 & \text{if } j = \arg\max_i \mathbf{w}_i^\mathsf{T}\mathbf{x} \\ 0 & \text{otherwise} \end{cases}$$

Single-layer neural network

- Non-linearity with all sums as input
  - Softmax function to output probabilities

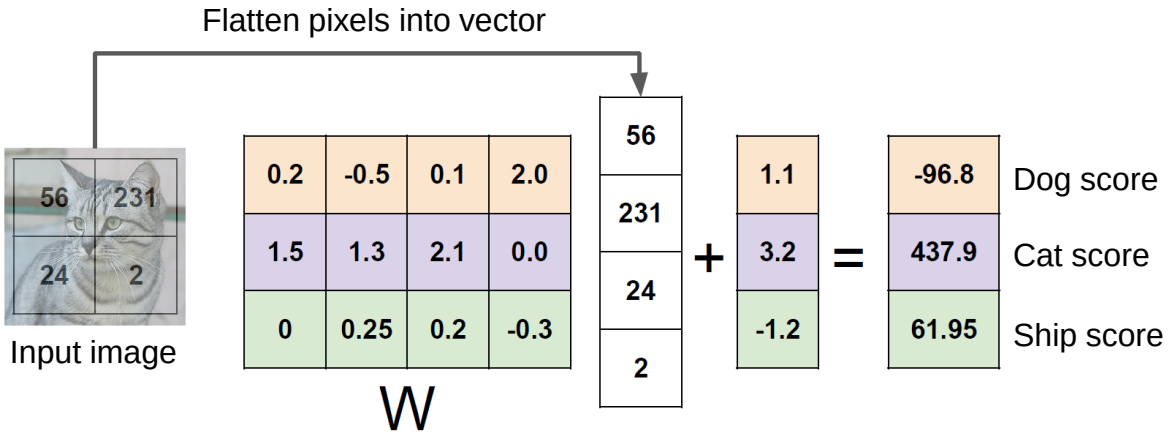


$$\mathbf{h} = [h_j]$$

$$h_j = g(\mathbf{s}) = \frac{e^{\mathbf{w}_j^T \mathbf{x}}}{\sum_i e^{\mathbf{w}_i^T \mathbf{x}}} \in [0, 1]$$

Multi-class linear classifiers

- Example: an image with 4 pixels, and 3 classes

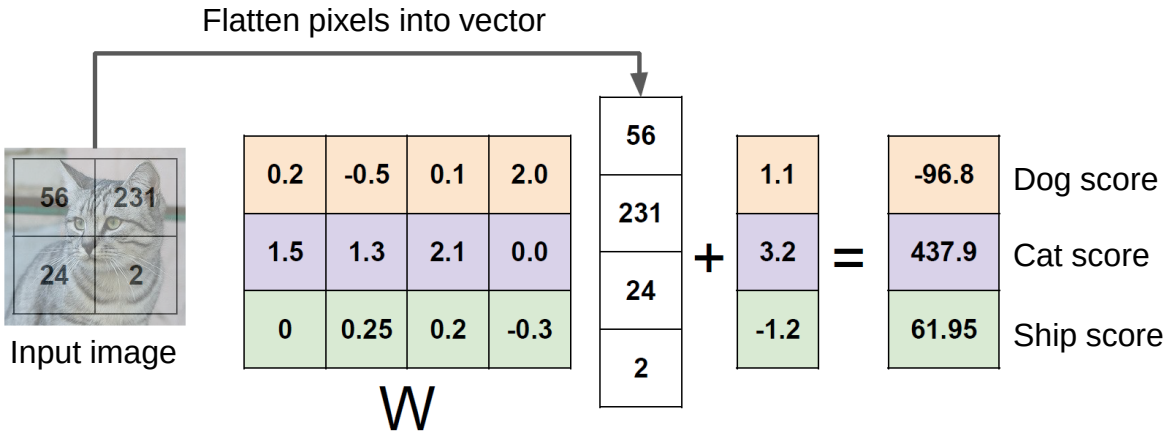


  - WTA prediction: one-hot encoding of class labels

$$y = 1 \Leftrightarrow y = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \qquad y = 2 \Leftrightarrow y = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \qquad y = 3 \Leftrightarrow y = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

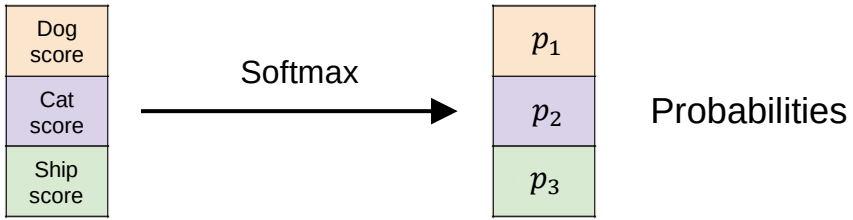
Multi-class linear classifiers

- Example: an image with 4 pixels, and 3 classes



- Softmax prediction: probabilistic outputs

Class logits or
Unnormalized log probabilities



How to learn a multi-class classifier?

- General learning problem setup
  - Given training data $\{(x_i, y_i)\}_{i=1}^n$
  - Find $f: x \to y \in \mathcal{H}$ that minimizes $L(f) = \frac{1}{n}\sum_{i=1}^n l(f, x_i, y_i)$
- Design a loss function for multi-class classifiers
  - Perceptron
    - See homework
  - SVM
    - Hinge loss
  - Logistic regression
    - Cross entropy loss (negative log likelihood)
- Avoid overfitting by regularization

How to learn a multi-class classifier?

- Minimize negative log likelihood of correct class

$$l(\mathbf{W}, x_i, y_i) = -\log p_{\mathbf{W}}(y_i|x_i), \quad p_{\mathbf{W}}(y_i|x_i) = \frac{e^{w_{y_i}^T x_i}}{\sum_j e^{w_j^T x_i}}$$

$$L(\mathbf{W}) = \frac{1}{n}\sum_{i=1}^{n} l(\mathbf{W}, x_i, y_i)$$

- Regularize weights

$$L(\mathbf{W}) = \frac{1}{n}\sum_{i=1}^{n} l(\mathbf{W}, x_i, y_i) + \lambda R(\mathbf{W})$$

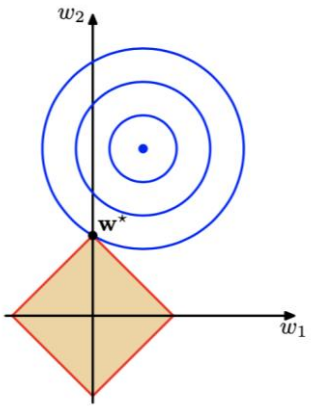
Regularization term
to reduce overfitting

Learning with regularization

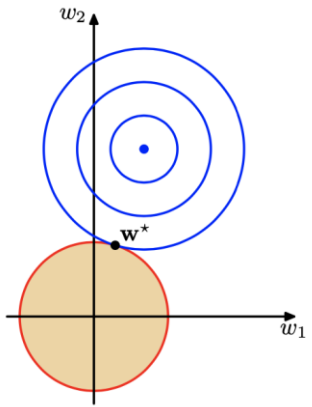
- L1 vs. L2 regularization

$$L(\mathbf{W}) = \underbrace{\frac{1}{n}\sum_{i=1}^{n} l(\mathbf{W}, x_i, y_i)}_{\text{Error}} + \underbrace{\lambda R(\mathbf{W})}_{\text{Regularization}}$$

L1: $R(\mathbf{W}) = \sum_{i,j} |\mathbf{W}_{ij}|$



Sparse weights



Small weights

L2: $R(\mathbf{W}) = \sum_{i,j} \mathbf{W}_{ij}^2$

Summary

- Perceptron algorithm
- Single-layer network

- Next time
  - Multi-layer neural networks
  - Computation in neural networks